



NEAR EAST UNIVERSITY
Faculty of Engineering
Department of Software Engineering

Double Exposure Generator

Graduation Project
SWE492

Omar Shanaa

Prof. Dr. Fadi Al-Turjman

Lefkosa-2026

Table of Contents

ACKNOWLEDGMENTS

ABSTRACT

INTRODUCTION

- 1.1 Problem Statement
- 1.2 Purpose and Objectives
- 1.3 Technologies Used
- 1.4 Scope of Project
- 1.5 Report Structure

LITERATURE REVIEW

- 2.1 Introduction
- 2.2 Review of Literature
 - 2.2.1 Double Exposure Photography
 - 2.2.2 AI-Based Image Segmentation
 - 2.2.3 Browser-Based Image Processing
 - 2.2.4 Real-Time Communication Technologies
 - 2.2.5 QR Code and Camera Technologies
 - 2.2.6 Research Gap

METHODOLOGY

- 3.1 SDLC Overview
- 3.2 Phase 1: Planning
 - 3.2.1 Project Scope
 - 3.2.2 Feasibility Assessment
 - 3.2.3 Risk Identification
- 3.3 Phase 2: Requirements Analysis
 - 3.3.1 Functional Requirements (FRs)
 - 3.3.2 Non-Functional Requirements (NFRs)

3.3.3 Data Requirements

3.4 Phase 3: System Design

3.4.1 High-Level Architecture

3.4.2 Data Flow Diagram

3.4.3 Rendering Pipeline Design

3.4.4 Security Design

3.5 Phase 4: Implementation / Development

3.5.1 Development Environment

3.5.2 Implementation Steps

3.6 Phase 5-6: Testing & Deployment

3.6.1 Testing Methods

3.6.2 Deployment Environment

3.6.3 Deployment Steps

RESULTS AND DISCUSSION

4.1 System Performance

4.2 Major Challenges to the Implementation

4.2.1 Browser Main Thread Blocking

4.2.2 MediaPipe Cold Start Delay

4.2.3 Camera Security Context Restrictions

4.2.4 Aply Realtime Synchronization Issues

4.3 Discussion

SYSTEM ARCHITECTURE & CODE STRUCTURE

5.1 Overall System Architecture

5.1.1 Client-Side Architecture Overview

5.1.2 System Components Diagram

5.2 Frontend Structure (User Interface)

5.2.1 HTML Structure

- 5.2.2 CSS Styling and Layout
- 5.2.3 JavaScript Modules
- 5.3 Image Processing Pipeline
 - 5.3.1 Web Worker Architecture
 - 5.3.2 OffscreenCanvas Rendering
 - 5.3.3 Blend Modes Implementation
 - 5.3.4 Cinematic Effects Pipeline
- 5.4 AI Segmentation Module
 - 5.4.1 MediaPipe Integration
 - 5.4.2 Mask Processing and Smoothing
 - 5.4.3 Luminance Fallback System
- 5.5 Smartphone Upload System
 - 5.5.1 QR Code Generation
 - 5.5.2 Ably Realtime Communication
 - 5.5.3 Session Management
- 5.6 Export and Camera Module
 - 5.6.1 Camera Capture Implementation
 - 5.6.2 Export Functionality
- 5.7 File & Folder Organization

DETAILED WORKFLOW EXPLANATION

- 6.1 Overall User Workflow
 - 6.1.1 Kiosk Session Start
 - 6.1.2 Image Upload and Capture Flow
 - 6.1.3 AI Segmentation Workflow
 - 6.1.4 Rendering and Effects Workflow
 - 6.1.5 Export and Download Flow

6.2 Smartphone Upload Workflow

6.2.1 QR Code Pairing Process

6.2.2 Image Transfer Process

6.2.3 Session End and Reset

6.3 Rendering Pipeline Workflow

6.3.1 Web Worker Communication Flow

6.3.2 Blend Mode Processing Steps

6.3.3 Cinematic Effects Processing Steps

CONCLUSION

References

ACKNOWLEDGMENTS

First, I would like to express my gratitude to Prof. Dr. Fadi Al Turjman for their support and guidance throughout this project.

I'm also thankful to Dr. Priscilla Olawale and Mr. Osman for their encouragement. Special thanks go to my friends and colleagues for their assistance.

And I extend my deepest appreciation to my family for their unwavering support throughout my academic journey, without which this project would not have been possible.

Abstract

Double Exposure Generator is a browser-based, AI-assisted image compositing kiosk designed to simplify the creation of artistic double exposure photography for non-expert users. Traditional double exposure photography typically requires advanced editing skills or darkroom experience, which makes it difficult for beginners and unsuitable for fast interaction in public settings. This project addresses that limitation through a fully client-side web application capable of generating cinematic double exposure artwork without backend infrastructure or native software installation. The system was developed using HTML5, CSS3, and JavaScript, with all image processing performed directly inside the browser. To maintain responsiveness during computationally intensive tasks, Web Workers and OffscreenCanvas were used to move heavy rendering off the browser main thread. Automatic subject isolation was implemented using MediaPipe Selfie Segmentation, which separates the foreground subject from the background to produce realistic blending. The application also provides a range of cinematic visual effects:

- grayscale conversion;
- ten professional blend modes for compositing;
- contrast enhancement;
- sepia toning for an analog film look;
- Gaussian film-grain simulation;
- light leaks, vignetting, and halation glow;
- and sharpening filters.

Users can transfer photographs from a smartphone by scanning a QR code.

Image delivery is handled over Ably Realtime channels, allowing photographs to be sent from a phone to the kiosk over a local Wi-Fi network. Testing showed that the application produces high-quality composites at interactive speeds and that the AI segmentation isolates subjects effectively, making the system well-suited to exhibitions, photo booths, and university events.

Introduction

1.1 Problem Statement

Double Exposure Generator is an app used to combine two pictures into one artistic picture. This technique has become very popular in digital photography. Producing professional double exposure art also requires advanced editing skills and knowledge of professional image editing software.

- Adobe Photoshop [1] is even a tool used for combining images, but it is too complicated to be used by normal people
- It is not good to be used in public places or events.

The existing solutions to create double exposure art on the web have many limitations: they require the installation of software, need an internet connection, or need a user account. These limitations make it difficult for users to create double exposure art at events, exhibitions, or in educational demonstrations. Additionally, traditional image editing systems are not intended for kiosk use, as they require users to interact easily, control simply, and see results immediately, with no technical knowledge required. Another challenge faced when processing multimedia within a browser context is that the required processing resources to perform tasks such as real-time image compositing, AI-based subject isolation, and creating high-resolution images consume a large amount of processing power, which may cause the interface to freeze if accessed through the main thread of the browser. Security rules imposed by browsers create challenges for applications designed for using a camera, as they restrict applications that attempt to use the `getUserMedia()` API to work only in secure contexts such as HTTPS and localhost. There is a need for a system designed to allow users to access the browser and produce high-quality double exposure art. This system must provide an easy-to-use interface, allow all functionality to run on the client-side without requiring backend technology, support high-performance processing, provide AI assistance for subject isolation, and support real-time image processing. This allows for a number of possibilities. The use of double exposure photography offers a great potential for creative expression and the ability to develop an interactive system to make creating exposures easier and more efficient for users. An effective interactive system should provide users with resources to allow them to create photographs with double exposures relatively quickly and easily.

1.2 Purpose and Objectives

The purpose of this project is to develop a browser-based, AI-assisted application that enables users to create double exposure photographs quickly and easily without requiring professional editing skills. The system simplifies image compositing through automated processing and an intuitive interface designed for non-technical users in public environments.

The primary objectives of the project are as follows:

1. To develop a browser-based image compositing system using HTML5, CSS3, and vanilla JavaScript without relying on external frameworks.
2. To implement AI-assisted subject segmentation for automatically isolating the foreground subject.
3. To create a responsive image processing pipeline that prevents the interface from freezing during heavy rendering, using Web Workers and OffscreenCanvas.
4. To support ten professional blend modes — Screen, Multiply, Overlay, Soft Light, Hard Light, Difference, Exclusion, Color Dodge, Color Burn, and Lighten — for artistic compositing.
5. To add cinematic enhancement effects such as sepia toning, film grain, vignetting, and halation.
6. To develop a smartphone upload feature using QR-code pairing and real-time communication.
7. To enable live camera capture through the MediaDevices API.
8. To provide image export in PNG, JPG, and WEBP formats, with an optional watermark.
9. To design an interface that is easy to operate for non-technical users in a public kiosk setting.
10. To demonstrate that advanced browser-native image processing can be achieved without backend servers or native software applications.

1.3 Technologies Used

The application has used modern web technology and browser APIs to facilitate real-time image processing via the client-side of web browsing. The application was designed specifically for use with Chromium-based browsers, including Google Chrome and Microsoft Edge. The application core structure was created using HTML5 for the page structure, CSS3 for responsive styles and visual effects, and vanilla JavaScript (ES2020) to implement application logic and handle interaction processing. No frontend development frameworks, such as React or Vue, were used in the application, so as to achieve lightweight deployment and minimize the runtime overhead of the application.

Image processing operations were implemented using the Canvas 2D API [11] and OffscreenCanvas [13]. Web Workers [12] were used to move computationally intensive rendering tasks away from the browser main thread, improving interface responsiveness during high-resolution image processing. ImageBitmap transferable objects were utilized for efficient zero-copy image transfer between threads.

Artificial intelligence functionality was implemented using MediaPipe Selfie Segmentation [10] loaded from a CDN through WebAssembly [14]-based execution. The AI segmentation pipeline enabled automatic subject extraction for realistic double exposure blending. Real-time communication between smartphones and the kiosk was implemented using the Ably Realtime platform [16] through WebSocket-based [15] publish/subscribe channels.

Additional technologies used in the project include ImgBB for temporary image hosting during phone transfer operations, Unsplash CDN images for texture galleries, QR code generation libraries [17] for mobile pairing, and Python's built-in http.server module for lightweight local hosting during kiosk deployment.

1.4 Scope of Project

This project delivers a browser-based kiosk application that generates AI-assisted double exposure artwork using fully client-side image processing. The application supports image upload, AI-based subject masking, multiple blend modes, cinematic visual effects, smartphone image transfer, camera capture, and high-resolution export. The scope is limited to browser-native technologies and does not include backend servers, database systems, or user-account management. All rendering is performed locally on the client device, and uploaded images are not stored permanently. The application was tested primarily on Windows 10 and Windows 11 using Chromium-based browsers. The smartphone upload feature operates over local Wi-Fi and depends on internet access for certain services, including MediaPipe CDN loading, Ably communication, ImgBB hosting, and Unsplash textures. Although most rendering can continue after initial loading, the current version does not fully support offline use. Advanced editing features such as manual layer masking, RAW editing, timeline-based composition, and multi-user collaboration are out of scope; the focus is on simplicity, accessibility, responsive performance, and suitability for interactive kiosk environments.

1.5 Report Structure

This report is organized into seven chapters covering the development, implementation, and evaluation of the proposed system.

The report begins with **Chapter 1**, which introduces the project background, problem statement, objectives, technologies used, project scope, and report structure.

Chapter 2 reviews the literature on double exposure photography, AI-based image segmentation, browser-based image processing, real-time communication, and web camera technologies.

The methodology is presented in **Chapter 3**, covering project planning, requirements analysis, system design, implementation, testing, and deployment.

Chapter 4 presents the results and discusses the major implementation challenges and how they were addressed.

The system architecture and code structure are described in **Chapter 5**, covering the frontend, image processing pipeline, AI segmentation module, and smartphone upload system.

Chapter 6 explains the detailed user and processing workflows, including the kiosk session flow, smartphone upload workflow, and rendering pipeline workflow.

Chapter 7 concludes everything explained in the report by summarizing the project's achievements, contributions, and future improvements for the app.

LITERATURE REVIEW

2.1 Introduction

As web-based technology, Artificial Intelligence, and Browser-based multimedia processing continue to advance rapidly, so has the way we develop interactive digital imaging systems. Previously, web browsers were limited to displaying only static webpages. Today’s browsers have been developed to perform advanced mathematical calculations online, real-time rendering of graphics (as well as general-purpose graphics processing units or GPGPU), access to cameras through the “WebBrowser” API, and multithreaded image processing. Thus, browser-based applications can perform many functions that previously required a dedicated desktop software application and/or a server-based processing architecture.

One of the most distinct forms of photographic compositing is ‘double exposure’ photography. Traditionally associated with analog darkroom methods, double exposure photography has now evolved into a digital image manipulation technique through the use of modern digital image editing software and mobile applications. However, although there is a wide variety of digital compositing tools available today, many require advanced knowledge of photo editing, software installation, and/or utilise cloud-based processing services. Thus, there continues to be a need for accessible, lightweight, and interactive systems that can create artistic photographic compositions in real-time.

Recent progress in artificial intelligence has also contributed to major improvements in image editing workflows. Deep learning-based image segmentation models can now isolate portrait subjects automatically with high accuracy, reducing the need for manual masking and selection operations. Frameworks such as MediaPipe [10] have further simplified browser deployment of AI models through WebAssembly [14] and optimized inference pipelines. At the same time, technologies such as Web Workers [12] and OffscreenCanvas [13] have improved browser performance by enabling computationally intensive rendering tasks to execute outside the main user-interface thread.

This chapter reviews previous research and technologies related to the development of the proposed system. The chapter examines the evolution of double exposure photography, AI-

powered image segmentation techniques, browser-based image processing technologies, real-time communication systems, QR-code device pairing methods, and browser camera APIs. The reviewed literature provides the theoretical and technical foundation for the implementation decisions used in the Double Exposure Generator project.

2.2 Review of Literature

2.2.1 Double Exposure Photography

Double exposure photography is one of the earliest artistic compositing techniques in the history of the medium [3]. In the analog film era, photographers had to expose the same frame multiple times, which required precise control of exposure balance, careful alignment of the images, and considerable manual experimentation in the darkroom. The transition to digital photography replaced these physical processes with software-based editing. Tools such as Adobe Photoshop [1] introduced layers, blend modes, and masking systems that allowed compositing operations to be performed digitally.

Although digital editing improved flexibility and creative control, professional compositing software remained difficult for non-expert users because of its complex interfaces and steep learning curves.

Mobile applications such as Snapseed and Lightroom Mobile later introduced simplified, AI-assisted compositing workflows [4]. However, many of these tools still rely on preset-driven workflows, cloud services, or mobile-only interfaces, and double exposure compositing remains a capability that users frequently want but cannot easily access.

There is therefore a continuing need for an accessible browser-based system that can produce double exposure effects through simplified interaction and automated processing, so that users do not need professional editing expertise.

2.2.2 AI-Based Image Segmentation

Image segmentation is a fundamental task in computer vision concerned with separating foreground objects from background regions within an image. Early approaches relied on traditional image-processing methods such as intensity thresholding [5], edge detection, and

region-growing algorithms. These techniques performed adequately on simple scenes but struggled with complex backgrounds and fine details such as hair boundaries.

The introduction of deep learning significantly improved segmentation accuracy through convolutional neural networks. Models such as Fully Convolutional Networks (FCNs) [7], U-Net [6], and DeepLab [8] adopted encoder-decoder architectures that capture high-level semantic information while preserving fine spatial detail, allowing both global context and local boundaries to be modelled simultaneously.

More recently, lightweight frameworks such as MediaPipe Selfie Segmentation [10] have enabled real-time foreground-background separation directly within a web browser, using compact, optimised neural networks based on MobileNetV3 [9] executed through WebAssembly [14] that run efficiently on consumer hardware. These advances make AI-assisted segmentation practical for browser-native, interactive multimedia systems.

2.2.3 Browser-Based Image Processing

Modern web browsers have become capable multimedia platforms. The Canvas 2D API [11] provides direct access to the individual pixels that make up an image, allowing rendering, filtering, and compositing operations to be performed entirely within the browser.

However, traditional Canvas rendering executes on the browser main thread, so intensive pixel operations can make the entire interface slow and unresponsive while the work is being performed.

This limitation is addressed by Web Workers [12], which allow JavaScript to run in parallel background threads, and by OffscreenCanvas [13], which allows rendering to execute independently of the visible interface.

In addition, transferable ImageBitmap objects enable zero-copy image transfer between threads, avoiding expensive memory duplication, while WebAssembly provides near-native execution speed for computationally demanding tasks such as AI inference. Together, these technologies make advanced browser-native multimedia processing possible without any desktop software installation.

2.2.4 Real-Time Communication Technologies

Real-time communication is essential for interactive web applications that require immediate data synchronization between devices. The traditional HTTP request-response model introduces significant latency and bandwidth overhead, making it unsuitable for highly interactive systems.

WebSocket technology [15] addresses these limitations by providing persistent, bidirectional communication channels between devices. Several managed platforms build on this foundation; Ably Realtime [16] is a cloud-managed service that provides scalable publish-subscribe messaging over WebSocket connections.

Ably Realtime is also straightforward to integrate because it provides client-side SDKs, removing the need for developers to manage their own servers, which distinguishes it from alternatives such as Socket.io and Firebase. Such real-time communication services are particularly well-suited to kiosk systems, where smartphones and public devices must exchange information instantly.

In the proposed system, Ably Realtime is used to enable QR-based smartphone upload and to synchronize state between the user's phone and the kiosk interface, allowing low-latency communication between the two devices.

2.2.5 QR Code and Camera Technologies

QR-code technology is widely used for device pairing, authentication, and mobile interaction with web applications. Standardized under ISO 18004 [17], QR codes can encode URLs, identifiers, and configuration data that are read by scanning them with a smartphone camera.

In kiosk systems, this significantly simplifies interaction, since users do not need to type network addresses manually, and their phones can connect to the kiosk directly.

Browser-based camera access has also improved through the `MediaDevices` and `getUserMedia()` APIs [2], which allow web applications to access device cameras directly from the browser.

However, browser security policy requires camera access to occur in a secure context, meaning the page must be served over HTTPS or from localhost.

This constraint creates a deployment challenge for browser-based kiosk systems, which must simultaneously support local camera access and network communication with smartphones.

Careful handling of these secure-context requirements is therefore essential when designing browser-based kiosk systems that combine QR-code pairing with camera access.

2.2.6 Research Gap

The reviewed literature demonstrates substantial progress in digital compositing, AI-based image segmentation, browser-based rendering, and real-time communication. However, most existing compositing systems still depend on native desktop applications or specialized hardware, and many remain difficult to use in public environments. Few systems combine automatic AI subject segmentation, real-time image compositing, smartphone-based upload, and kiosk-oriented usability within a single browser-native application. This represents a detailed research and engineering gap. The proposed project addresses this gap by integrating AI-assisted subject segmentation, fully client-side image processing, and real-time smartphone interaction into a lightweight, self-service kiosk platform that is accessible to non-expert users, suitable for public interactive environments, and capable of performing well on modest hardware.

Methodology

3.1 SDLC Overview

The Double Exposure Generator project was developed using an approach. This approach is called the Software Development Life Cycle methodology. It helps with planning, doing, testing and putting the application to use.

The Double Exposure Generator project utilized many elements of technology (web-based image processing, AI segmentation, real-time communication systems and interactive kiosks) which required a managed approach to ensure all components worked together seamlessly.

The project was executed in parts, tested and then combined into an integrated final package. The SDLC (Software Development Life Cycle) was used to provide the framework for each of the components. The major components of the Software Development Life Cycle include Planning, Requirements Analysis, System Design, Implementation, Testing and Deployment. Each of these components reduced overall risk and improved the quality of the final software product.

The use of the Software Development Life Cycle methodology helped the project in several respects; 1) the project was organized much better since the system was developed in parts and as individual modules; 2) debugging and testing was much more efficient because each part was tested individually prior to putting into production with the other parts; 3) the system was designed in such a way that it would be capable of future expansion and would require less maintenance; and 4) throughout the entire project the Software Development Life Cycle methodology ensured that the technical requirements of the project were satisfied as well as providing a good overall experience for all users. The Double Exposure Generator project and Software Development Life Cycle methodology worked very well together.

3.2 Phase 1: Planning

3.2.1 Project Scope

The objective of this project was to develop a computer program that allows users to create beautiful double-exposure images right on their web browsers without the need for any third-party software or equipment on the server side. The program finds the two images and places them side by side, allowing for a double exposure effect.

In addition to finding the images to place next to one another, the program has many features. It allows users to upload images of themselves and provides a feature to help users cut themselves out of their original background. The program also has the capability of adding to the user's images any number of textures or effects, allowing the final image to look like artwork. This feature is functional for images taken from a user's mobile device, but also can be taken directly from the user's computer with a camera. When you are done you can save the picture.

This kind of application is great for places like museums, universities and photography booths where people like to take fun pictures. The project only used things that web browsers can do. It only worked on the computer not on any big servers.

Some things that people might want, like cloud accounts to store their pictures online or working on them together with other people were not part of this project. The application worked well on computers that run Windows and use the Chromium browser like Google Chrome.

3.2.2 Feasibility Assessment

The people in charge did a study to see if the proposed system would actually work before they started building it. They looked at the part of the proposed system to see if it was viable.

The proposed system is viable from a point of view because of new browser technologies like Canvas API and Web Workers and OffscreenCanvas and MediaPipe AI segmentation.

These technologies made it possible to process images in the browser without needing to use the server. Previous studies on systems that use browsers to handle multimedia showed that modern browsers can handle image operations if they use multithreaded rendering techniques.

The proposed system is also viable from a point of view. This is because the workflow for the kiosk was designed to be easy for people who're not technical to use.

Using QR codes to upload from smartphones made it simpler for people to interact with the system and got rid of the need to use things like USB drives to transfer files. The proposed system is viable, from a point of view too. This is because the project uses open web technologies for the most part. The proposed system only needs a laptop with a modern browser to work and does not need special servers or expensive software licenses.

3.2.3 Risk Identification

There were considerable operational risk factors resulting from the planning process. A significant risk was the inability of some browsers to effectively handle high-resolution images. The use of high-resolution images requires an extensive amount of processing as working with high-resolution images in real time, combined with the separation of images using artificial intelligence, will consume a significant amount of the computer's central processing unit (CPU) and random-access memory (RAM). The resulting freeze in the user interface was resolved by utilizing Web Workers and OffscreenCanvas rendering. Another major risk occurred due to security protocols established by the browsers that limited access to the user's camera. For example, the `getUserMedia` API will only work on a web application that is hosted within Secure Hypertext Transfer Protocol (HTTPS) or running on the localhost.

To counter this risk, a localhost server was configured using a special URL that allowed for communication with QR Code readers. The team also knew that the system would not work well if it was not connected to the internet. This is because some services, like MediaPipe and Ably need the internet to work. So, they came up with plans like using luminance-based masking to make the system less dependent on artificial intelligence. There were some risks too like the system not working on all browsers or the internet connection being bad when you try to upload something from your phone.

The team was also worried that the system might use up much memory when you try to export high-resolution images. The system had risks like browser compatibility issues and unstable Wi-Fi communication during phone upload and potential memory consumption, during high-resolution export operations of the system.

3.3 Phase 2: Requirements Analysis

3.3.1 Functional Requirements (FRs)

The system has to do things. It must let users upload pictures of people and textures into layers. The application has to use intelligence to separate the person in the picture automatically. Users have to be able to try ways of combining pictures and add special effects to make double exposure pictures.

The system must let users drag and drop pictures, see a live preview, choose from presets, make the picture brighter or darker, add a grain effect, and add a light-leak effect. The application must also let users upload pictures from their smartphone using a QR code and communicate with the system in real time.

There are some things the system has to do. It has to let users take pictures with their browser compare the before and, after pictures save the picture in quality add a watermark and reset automatically if nobody is using it for a while in a public computer setting. The system must support all these features to work.

3.3.2 Non-Functional Requirements (NFRs)

The system has to work and be easy to use. This means it has to be fast and not slow down when it is doing things.

The application has to keep working when it is doing rendering operations. It should not freeze the user interface.

Rendering operations have to finish on regular computers that people buy.

The system has to work with browsers like Google Chrome and Microsoft Edge.

When it comes to usability requirements for the system, we need to make sure the system is easy to use.

This means the system has to have ways for users to interact with it.

The system has to give users signs that it is working.

The system has to be designed in a way that's easy for people who are not good with technology to use.

Security requirements, for the system are also important.

This means the system has to have rules that limit what content can be used.

The system has to control how the camera is used.

It has to make sure the browser is secure when it is using the camera.

The application has to be made in a way that's easy to maintain.

This means the JavaScript code has to be organized in a way.

The system has to have rendering components that can be used again.

This will make it easier to keep the system working and fix problems when they happen.

3.3.3 Data Requirements

The system is used to process mainly multimedia data rather than permanent data stored as permanent records in a database; Therefore, the system has been developed to work with data extracted from image files (e.g. a photo taken with a camera) as well as Mask types generated from Image Extraction or rendering, or export types used to generate an exported version of the same image. Uploaded image files

- camera captures
- artificial intelligence segmentation masks
- rendering settings
- export configurations

The application state, which is the relationship between the components being blended together (i.e. where each of the component slider positions are), what effect was applied to each of them, and what preset was selected when the components were blended, is only temporary and is stored in Local Storage on the user's browser. The system does not keep user images after they are uploaded which helps keep things and saves space. When the kiosk session starts over. The page is loaded again all the data used for rendering is removed. This means the system gets rid of all the

data it was using which includes the user images and other things. The system is designed to work with multimedia data like the uploaded image files and artificial intelligence masks and it does not store this data permanently.

3.4 Phase 3: System Design

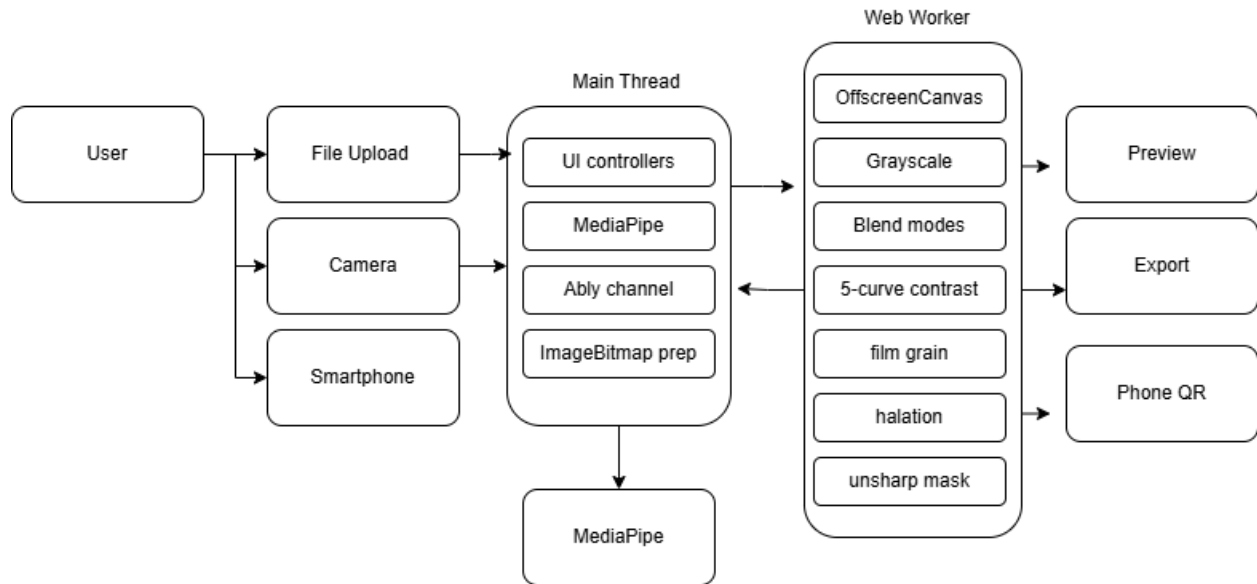
3.4.1 High-Level Architecture

Connected via Wi-Fi, the System consists of a laptop with a mobile phone network linking to two telephones (smartphones) and a number of laptops. The laptops and smartphones function together as a team; they use the laptop to accomplish tasks while linking to and from the laptop to accomplish those tasks.

The public utilizes the laptop as a means of accessing the Internet, while the smartphones communicate to the laptop using a QR Code. While the laptop carries out much of the heavy lifting associated with programming and processing data (e.g., generating large files (images) etc.), it will delegate some of these heavy tasks via use of the Web Worker technology to ensure continuous operation and performance. The laptop also runs the AI part that figures out where the person is in a picture, but it has to do this on the main thread because MediaPipe does not work well inside a Web Worker. The system uses MediaPipe Selfie Segmentation for this, and it runs in the browser on the laptop.

3.4.2 Data Flow Diagram

The system data flow starts when the user uploads or takes a picture. This picture is then changed into an ImageBitmap object. Sent to the rendering pipeline. If AI masking is turned on MediaPipe segmentation creates a foreground confidence mask that is given to the rendering worker. The rendering worker does a lot of things to the picture it makes it grayscale blends it makes the contrast better adds some grain to it. Gives it an artistic effect. After all these things are done the worker sends the picture back, to the main thread. Then this final picture is shown to the user and the user can save it as a PNG, JPG or WEBP file if they want to. When someone uses their Smartphone to upload a picture it goes through a path. The phone sends the picture to the kiosk browser through something called Ably Realtime channels.



3.4.3 Rendering Pipeline Design

The rendering pipeline is made up of a sequence of stages that work together to process the image. Each stage performs one specific operation: grayscale conversion of Layer I using the ITU-R BT.709 luminance formula ($\text{luma} = 0.2126R + 0.7152G + 0.0722B$) [19], optional subject masking, the selected blend operation, an S-curve contrast adjustment that boosts midtone separation without crushing shadows or clipping highlights, a 3x3 sepia matrix, Box-Muller film grain, light leaks, vignetting, unsharp-mask sharpening (luma minus a box-blurred luma, scaled by strength), halation glow, and chromatic aberration. All color calculations are performed in normalized floating-point space [0,1] so colors stay accurate during blending. The pipeline executes inside a Web Worker on an OffscreenCanvas, so this work happens off the main thread and the interface stays responsive. The modular stage design also makes the pipeline easy to extend, since new effects or blend modes can be added as additional stages without changing the rest of the pipeline.

The pipeline is made this way so it is easy to add things to it later like new visual effects or ways of blending pictures and these new things can be added one at a time without messing up the rest of the rendering pipeline.

3.4.4 Security Design

The application has security measures to keep users safe when they use it and to protect the devices they use.

It enforces a strict inline Content Security Policy that restricts scripts, styles, images, and network connections to a known set of trusted sources only (the application's own origin, the jsDelivr and Ably CDNs for the MediaPipe and Ably libraries, ImgBB and Unsplash for image transfer and textures, and the Ably WebSocket endpoints). The 'unsafe-eval' directive is permitted only because the MediaPipe WebAssembly runtime requires it, and 'worker-src blob:' is required to allow the inline Blob-embedded Web Worker.

This means that the application can only connect to the internet if it is supposed to.

When you use the camera on your device it will only work if you are on a connection and you are using the application on your own device.

The application uses codes to talk to other devices and it uses random identifiers so that only the right devices can talk to each other.

The application uses keys to talk to other services like Ably and ImgBB and it puts these keys in a special code when it makes a QR code.

The system is designed to be used on a network, which means it is not meant to be used on the public internet so it assumes that the network it is on is safe.

This is because the application is meant to be used for events and things like that and it will not be, on the internet all the time.

3.5 Phase 4: Implementation / Development

3.5.1 Development Environment

The project was built using Visual Studio Code as the development environment. Testing was done mostly on Windows 10 and Windows 11 computers with Google Chrome and Microsoft Edge browsers, which worked well for testing. The application was built with HTML5, CSS3 and plain

JavaScript, with no frontend frameworks. The use of python's http.server module to facilitate local hosting made it easier to develop and release this project.

3.5.2 Implementation Steps

Step 1: Core Browser Architecture Setup

Creating a page for users to use was the first step towards building this project. This was done using HTML, CSS and JavaScript. We made areas where users can upload things. We added sliders and modal dialogs to make it easy to use. We also added controls and containers to show the things that are being rendered. The single-page kiosk structure was the focus of this initial development stage and we used HTML, CSS and JavaScript to make it work.

Step 2: AI Segmentation Integration

MediaPipe Selfie Segmentation was added to automatically extract the foreground from portrait images. This feature helps to separate the subject from the background. Mask smoothing, blur and luminance fallback were also implemented. These improvements were made to enhance the performance of MediaPipe Selfie Segmentation. The goal is to make MediaPipe Selfie Segmentation more effective for use, in portrait images.

Step 3: Rendering and Image Processing Pipeline

The Web Worker rendering pipeline was developed to handle grayscale conversion, blend modes, film grain, vignette effects, halation glow, sharpening, and chromatic aberration. OffscreenCanvas was integrated to improve rendering performance.

Step 4: Real-Time Communication and Export Features

We made a feature for smartphones that uses QR codes and Ably Realtime communication channels. This feature lets people upload things from their phone. We also worked on export pipelines so people can save things as PNG, JPG and WEBP files. In addition, we made a way to add watermarks to these files and a special QR code that sends files to people's phones.

3.6 Phase 5–6: Testing & Deployment

3.6.1 Testing Methods

The system was tested using manual functional testing, browser compatibility testing, performance evaluation, and user interaction testing. Functional testing verified image upload, AI masking, blend modes, camera access, QR upload, export operations, and preset functionality. The rendering performance evaluation includes web worker efficiency and responsiveness when running at high-quality rendering resolution. Most of the cross-browser tests (for example, through OffscreenCanvas) used have been limited to Chromium-based browsers as they supported OffscreenCanvas.

3.6.2 Deployment Environment

The application has been designed to operate using browsers such as Google Chrome or Microsoft Edge on typically used Windows-based laptop systems. To use it you need to have a Wi-Fi network so your smartphone can talk to it and you can upload things using QR codes. The system works completely in your browser so you do not need servers or databases to make it work. You can just use the application in Google Chrome or Microsoft Edge, on your Windows laptop.

3.6.3 Deployment Steps

To deploy the system, the `run_studio.bat` script is run. This script performs several steps: it detects the active LAN IPv4 address on the machine using PowerShell commands, starts a local web server using Python's built-in `http.server`, and opens a browser window at `localhost:8000`. The browser receives the LAN IP through a `?lanip=` URL parameter, which is used to build the QR code and connect smartphones to the system. The kiosk itself runs at `localhost` (rather than the LAN IP) because the camera API requires a secure context, while the phone QR code points to the LAN IP so the phone can reach the server. As an additional robustness measure, the image-processing Web Worker is embedded as a Blob URL rather than loaded from a separate file, so the application also works correctly when opened directly as a `file://` URL, which Chrome would otherwise block for external Web Worker files. Users can connect their phones to the system by scanning the QR code.

After that they can send pictures from their phones to the system over the network. The kiosk system will get these pictures directly.

```
1 @echo off
2 echo Starting Double Exposure Darkroom Studio...
3 echo.
4
5 :: Find the IP of the adapter that has a default gateway (the real WiFi/Ethernet, not virtual adapters)
6 for /f %% in ('powershell -nologo -noprofile -command "$idx=(Get-NetRoute -DestinationPrefix '0.0.0.0/0')|Sort-Object RouteMetric|Select-Object -First 1).InterfaceIndex;(Get-NetIPAddress -InterfaceIndex $idx -AddressFamily IPv4).IPAddress') do set LOCAL_IP=%%
7
8 if "%LOCAL_IP%"=="" (
9     echo WARNING: Could not detect your WiFi IP. QR code may not work on phone.
10    set LOCAL_IP=localhost
11 )
12
13 echo -----
14 echo Laptop URL : http://localhost:8000
15 echo Phone URL  : http://%LOCAL_IP:8000
16 echo -----
17 echo Open the Phone URL on your laptop to make
18 echo the QR code work on your phone's camera.
19 echo -----
20 echo.
```



```
1  async function processImages() {
2    return new Promise((resolve, reject) => {
3      // Create a Blob URL from the inlined worker source so it works
4      // even when the page is opened as a local file:// URL.
5      const blob = new Blob([WORKER_SOURCE], { type: 'application/javascript' });
6      const blobUrl = URL.createObjectURL(blob);
7      const worker = new Worker(blobUrl);
8
9      worker.onmessage = (e) => {
10        URL.revokeObjectURL(blobUrl); // clean up
11        if (e.data.error) {
12          reject(new Error(e.data.error));
13        } else {
14          if (window._lastResultUrl) URL.revokeObjectURL(window._lastResultUrl);
15          resolve(URL.createObjectURL(e.data.blob));
16        }
17        worker.terminate();
18      };
19
20      worker.onerror = (err) => {
21        URL.revokeObjectURL(blobUrl);
22        reject(err);
23        worker.terminate();
24      };
25
26      // Prepare AI mask buffer (transfer if available)
27      const aiMaskCopy = (state.aiMask && state.useMask) ? new Float32Array(state.aiMask.data) : null;
28      const aiMaskWidth = state.aiMask ? state.aiMask.width : 0;
29      const aiMaskHeight = state.aiMask ? state.aiMask.height : 0;
30
31      // Clone the ImageBitmaps so originals stay usable after transfer
32      Promise.all([
33        createImageBitmap(state.img1),
34        createImageBitmap(state.img2),
35      ]).then(([bmp1, bmp2]) => {
36        const transferList = [bmp1, bmp2];
37        if (aiMaskCopy) transferList.push(aiMaskCopy.buffer);
38        worker.postMessage({
39          img1: bmp1,
40          img2: bmp2,
41          aiMaskData: aiMaskCopy,
42          aiMaskWidth,
43          aiMaskHeight,
44          state: {
45            blendMode: state.blendMode,
46            opacity: state.opacity,
47            contrast: state.contrast,
48            brightness: state.brightness,
49            sepia: state.sepia,
50            grain: state.grain,
51            leak: state.leak,
52            halation: state.halation ?? 0.10,
53            sharpness: state.sharpness ?? 0.00,
54            useMask: state.useMask,
55            highRes: state.highRes,
56            overlayX: state.overlayX,
57            overlayY: state.overlayY,
58            overlayScale: state.overlayScale,
59          },
60        }, transferList);
61      }).catch(reject);
62    });
63  }
```

4.1 System Performance

The Double Exposure Generator system was effective in displaying images together in real time via the web. It utilized a web technology that runs on the user's local computer rather than through a web server. The technology demonstrated that it was capable of producing high-quality double exposure images at a relatively fast rate (with limited user interaction). It utilized Web Workers and OffscreenCanvas to provide high-performance processing of image data by processing intensive image processing tasks outside of the main thread of the web browser.

For interactive kiosk applications, the standard export resolution is limited to a maximum of 900 pixels in the longest dimension, while generating an image with a display resolution of 300-800 milliseconds (varying based on how complex the image is, which filters/effects were applied, and performance of the web browser). High-resolution export performance is significantly slower than standard export performance due to the fact that high-resolution exports require more memory and processing time per pixel because they render at the size of the final image.

Through its ability to produce high-quality images, the MediaPipe Selfie Segmentation model was useful for generating portrait images (effects) with clean edges while removing them from other aspects of the environment. Once the model was loaded, typical CPU segmentation processing times were between 300-800 ms. Smoothing the edges of the mask and applying effects to the mask improved quality and reduced visibility of defects along the edges around hair and facial features. The QR-code based system for uploading photos from mobile devices also worked well on Wi-Fi networks. Users could upload photos from their mobile devices directly to the self-service kiosk without having to touch the kiosk laptop.

In order to keep the synchronous communication system in sync and minimize delay, the image transfer and session control were kept fast.

The user interface remained responsive throughout testing as all image rendering occurred in a worker thread.

Buttons, sliders and drag interactions functioned seamlessly in parallel to the large amount of image processing occurring in the background. Using ImageBitmap objects that can be transferred

reduced the need for multiple copies of images across various threads and optimized memory usage.

The Double Exposure Generator created a way for compositing images in real time with an easy-to-use interface. It combines the use of Web Workers and OffscreenCanvas (two new technologies) to help reduce processing time and improve performance by allowing complex image-processing jobs to be carried out off of the primary browser thread.

The MediaPipe Selfie Segmentation functioned well for isolating portraits. The QR-based smartphone upload provided a way for users to upload their images over local Wi-Fi networks.

Overall, the project met the goals of the project, which included, an end product that operates well in a web browser; creating a product that uses AI (artificial intelligence) to provide assistance in allowing for the creation of composite images on smartphones; and creating a product that would work well on kiosks. The end result of the project demonstrated that the latest web browsers can perform well with multimedia functions without the need for additional software applications or support from an external server to perform these functions. The final application demonstrated that browser-native render engines and AI-assisted compositing are compatible; and provide a means to integrate both with mobile and kiosk usability.

Feature	Average Performance	Observation
Standard Rendering (900px)	1–3 seconds	Responsive during rendering
High-Resolution Export	4–10 seconds	Depends on image size
MediaPipe Segmentation	300–800 ms	Faster after first load
QR Phone Upload	1–5 seconds	Depends on Wi-Fi speed
Export Generation	<2 seconds	PNG slower than JPG

4.2 Major Challenges to the Implementation

4.2.1 Browser Main Thread Blocking

The big problem we had when we were making this thing was that the browser would freeze up.

This happened because the computer had to do a lot of work to process the images.

We were using the Canvas API to draw the pictures on the screen.

But when we did things like turn the pictures to black and white or add effects,

it took a lot of calculations and made the browser slow down.

The browser would basically lock up. You could not use it while it was doing these things.

This was a problem because we wanted people to be able to use this thing with a special computer that you can touch.

When we used pictures, it was even worse because the computer had to do even more work.

The sliders and buttons and other things you can touch on the screen would not work while the pictures were being processed.

To fix the problem, Web Workers and OffscreenCanvas were used, and this lets you work with images from behind the scenes so when you work with them the browser won't be slow or lock up.

The other area we looked at was Instructional Design, specifically in a way we could transfer images with less memory than we were using before. This resulted in significant overall performance gain due to both implementation of these two pieces; taking the image processing out of the interface and putting it into the background, allowed for the user to still be able to use the interface while the images were rendering at peak times plus it still had the same type of speed throughout the whole process.

Challenge	Cause	Solution
Main Thread Blocking	Heavy pixel operations	Web Workers + OffscreenCanvas
MediaPipe Cold Start	WASM model loading	Input downsampling + fallback mask
Camera Restrictions	Secure context policy	localhost + ?lanip= strategy
Sync Issues	Network instability	Session management + reconnect handling

4.2.2 MediaPipe Cold Start Delay

Another big challenge was with the MediaPipe Selfie Segmentation framework. We initially had challenges due to downloading models used in processing media, along with downloading parts of WebAssembly modules used in the browser from different locations. Beginning the segmentation of an object was very slow because the following had to occur first: 1- download the model files through the browser, 2- create a WebAssembly run-time environment to run the model and 3- allocate the memory so that the model used and the image inputs worked together easily. Because of this, customers experienced slow performance from the application. Of the three problems I just described above, the slowest was for users with slow internet connections and/or older computers. To alleviate these problems, we reduced the size of images used as inputs so there was less processing done prior to segmentation. We implemented a system of ensuring that all segmentation requests were satisfied sequentially by incorporating a queue of requests matched to promise objects; therefore, eliminating any chance for conflict when the models were run. Lastly, we created a fallback masking system in which, if MediaPipe is not able to be loaded or no internet connection exists, the application would function properly. While we could not completely eliminate the delay from the time you began segmentation until the time you finished, the enhancements we implemented allow for the application to be more responsive to input and stable than originally could be expected while many customers utilized the kiosks and/or operated with inferior computing platforms.

4.2.3 Camera Security Context Restrictions

Browser camera capture created another challenge to browser security policies. Today, modern browsers block camera use unless the page is running in a secure environment (HTTPS or localhost). This creates problems for our kiosk because the kiosk has to be accessible on the network so that users can upload from their smartphones; however, the kiosk can only use the camera when the kiosk is running on the localhost. Therefore, if users type the LAN IP address of the kiosk using an unencrypted HTTP connection, the browser will deny camera access. This made the kiosk difficult to set up because both the camera and the smartphone need to communicate with each other at the same time. The problem was solved by using a URL parameter technique in the kiosk's startup script: The kiosk runs on localhost:8000 to allow for camera use, and the

detected LAN IP address is passed to the kiosk via the ?lanip= URL parameter to create the QR code used for smartphone upload. In conclusion, this emphasizes the importance of understanding browser security policies when developing web applications that utilize the camera. Proper implementation of browser security policies was crucial to the implementation of kiosks.

4.2.4 Ably Realtime Synchronization Issues

The smartphone remote upload system utilizes Ably Realtime communication channels for synchronizing devices and facilitating message transfer. While developing this system we encountered several issues with synchronization. Problems arose when the network was unstable, resulting in continuous re-connections of sessions. One of the problems we encountered was that disconnected smartphone sessions could reconnect with previously used QR codes. This resulted in users having the ability to continue using channels that were no longer valid following the resetting of the kiosk. As a result, synchronization problems may have occurred or unanticipated uploads may have taken place. The system also experienced timing issues with message arrivals. A large volume of uploads would occur or the system would repeatedly attempt to reconnect, causing delays in message arrival when the Wi-Fi connection was unstable. Consequently, the phone interface and the kiosk display would be out of sync. To resolve these problems, we used randomized session identifiers, utilizing the kiosk ID, and temporarily stored the state of disconnections within the browser session storage. Upon disconnection, the Ably communication channel would be closed at the Kiosk and the smartphone interface would present a message indicating that the session had ended to prevent individuals from continuing to use expired sessions. After these changes the smartphone upload system worked better and had fewer synchronization errors when we tested it in the real world.

Section 4.3—Discussion

The final implementation of the system provides evidence of the capabilities of modern browsers with respect to multimedia usage and AI applications. Applications requiring real-time image processing, computer vision and interactive rendered images often required specific software on a local machine or powerful graphics processors available at the server. The results of this project show that modern browser technology can perform very complex artistic tasks without any assisting software on the local machine and solely reliant on browser capabilities.

The other significant discovery from this project was that combining Web Workers with OffscreenCanvas rendering allows the application to remain interactive while performing complex image processing requiring extremely high-computing power. The use of rendering using multiple threads has significantly improved usability and has demonstrated that well-designed applications can overcome the limitations imposed by browser performance.

The MediaPipe Selfie Segmentation tool made it possible for people who are not specialists to create professional-looking work by automatically removing the background from an image while simplifying the process of creating artistic composites. Automated background removal reduced the need for a person to manually remove the background, allowing anyone to quickly produce exposure artwork that looked good. Even though the initial process of using AI to segment images was time-consuming and sometimes does not produce precisely aligned edges, combining AI with various smoothing and other methods to get acceptable results for commercial kiosks was sufficient. The ability for individuals to upload images from a phone made it easy for those people using the device to interact without physically touching the kiosk. Additionally, connecting devices and synchronizing them in time using QR codes reduced the complexity of interaction and facilitated rapid transfer of images for use in demonstrations and testing.

Despite the project being a success, there are still several limitations to the current project. For example, the application must be connected to the internet in order to access and utilize Artificial Intelligence trained models and communicate with other web services. Furthermore, various browsers interpret and render OffscreenCanvas and other advanced rendering technologies differently.

In addition, the current implementation could be improved by integrating Artificial Intelligence training models into the application to enable it to function offline and supporting additional types of image file formats and composite workflows. Overall, this project would support future development of an education/commercially usable, lightweight, browser-based, A.I., powered, compositing kiosk for creative, educational, or publicly interactive environments. As such, it also demonstrates that modern browsers represent a viable platform for advanced multimedia (i.e., video) and A.I. applications, and that present browsers demonstrate the continued evolution in processing power of modern browsers.

System	Browser-Based	AI Segmentation	Phone Upload	Offline-Capable
Photoshop	No	Limited	No	Yes
Snapseed	No	Partial	No	Yes
Proposed System	Yes	Yes	Yes	Partial

SYSTEM ARCHITECTURE & CODE STRUCTURE

5.1 Overall System Architecture

The system operates on a web-based application, meaning that it runs on an end user's computer and does not require any server infrastructure or special software. The application can create images and perform various functions while not affecting performance levels of the web browser itself. The application has been developed to be highly responsive and easy to use via a kiosk, allowing it to complete many demanding tasks without affecting the operation of the interface, allowing the interface to run smoothly. The application is able to communicate with mobile phones via instant messaging channels in real-time so that users of mobile devices can upload images without requiring interaction with the kiosk. The application is composed of four main components; the user interface (UI), the rendering engine (RE), the artificial intelligence (AI) segmentation engine, and the upload configuration module (UCM). Each of the four components works together to enable users to create double exposure images quickly and in real-time when working with this application.

5.1.1 Client-Side Architecture Overview

The application is built using a single page design with HTML5, CSS3 and basic JavaScript. The browser does all the work it shows the pages. It runs all the system logic. This is different from web applications that need a server to do the work. The proposed system does all the image work locally in the browser. The system has two parts. The first part is the browsers thread it handles what the user does it shows the interface it does the AI work it gets the camera pictures and it handles communication. The second part is the Web Worker thread it does the work of processing images. When users add or take pictures the browser changes the pictures into something it can use before sending them to the worker. The worker then processes the pictures. Sends the final result back to the main thread to show and save. This design makes the application more responsive, by keeping the interface and rendering work separate. So the kiosk interface still works when it is doing big image work in the background. The application stays interactive. Users can keep using it while the image work is being done. This makes the application better because the kiosk interface does not freeze when it is doing image work. The image work is done by the Web

Worker thread and the main thread keeps the interface working. This is why the application is more responsive and better to use.

5.1.2 System Components Diagram

This system is comprised of the core functional parts.

1. The User Interface Layer is the graphical representation of what you see and operate on the system. There are areas in the User Interface Layer where you can upload files, sliders for blending images, buttons for using preset effects, as well as pop-up windows displaying results.
2. The AI Segmentation Module utilizes the MediaPipe Selfie Segmentation process to isolate individuals in photo images (pictures) and create masks around them.
3. The Web Worker Rendering Engine performs many of the functions within the system; it assembles the image files, creates special effects, blends images together and prepares the output to be sent out.
4. The Smartphone Upload System provides communication between your smartphone device and the overall system. It is accomplished through the use of QR Codes, the Ably Realtime service and provides you the ability to upload photos from your smartphone to the overall system.
5. The Camera Capture Module utilizes the MediaDevices Application Programming Interface (API) to allow you to use your webcam to take and upload photos.
6. The Export Module provides you the ability to save exported photos as either a PNG, JPG or WEBP file format. Additionally, you will also be able to add a watermark if you choose.
7. The state management system manages your personal settings for rendering, effects, presets, and all other settings necessary to preserve the position of the controls (the state of the images you have uploaded to the server will not be preserved because they are temporary). The state management system uses browser localStorage to save the chosen

rendering properties, effect values, and chosen preset, which will keep the position of controls across reloads.

PRESETS: The state management system has seven named style presets that the user can choose from. Each preset comes with all of the rendering properties.

Classical - Includes: Typical screen blend, warm sepia tone, moderate amount of grain.

```
1 const PRESETS = {
2   classic: { blendMode: 'screen',    opacity: 0.65, contrast: 1.10, brightness: 1.00, sepia: 0.30, grain: 0.20, leak: 0.15, halation: 0.10, sharpness: 0.00 },
3   forest:  { blendMode: 'overlay',   opacity: 0.72, contrast: 1.20, brightness: 0.95, sepia: 0.05, grain: 0.15, leak: 0.08, halation: 0.06, sharpness: 0.15 },
4   ghost:   { blendMode: 'lighten',   opacity: 0.55, contrast: 0.95, brightness: 1.10, sepia: 0.00, grain: 0.10, leak: 0.25, halation: 0.20, sharpness: 0.00 },
5   noir:    { blendMode: 'multiply',  opacity: 0.80, contrast: 1.35, brightness: 0.85, sepia: 0.60, grain: 0.35, leak: 0.05, halation: 0.04, sharpness: 0.25 },
6   golden:  { blendMode: 'soft-light', opacity: 0.70, contrast: 1.15, brightness: 1.05, sepia: 0.50, grain: 0.12, leak: 0.20, halation: 0.15, sharpness: 0.10 },
7   cosmic:  { blendMode: 'color-dodge', opacity: 0.40, contrast: 1.20, brightness: 0.88, sepia: 0.00, grain: 0.08, leak: 0.32, halation: 0.30, sharpness: 0.05 },
8   dusk:    { blendMode: 'difference', opacity: 0.55, contrast: 1.05, brightness: 1.00, sepia: 0.15, grain: 0.18, leak: 0.20, halation: 0.14, sharpness: 0.00 },
9 };
```

Forest - Includes: Overlay blend, cool, high contrast.

Ghost - Includes: Lighten blend, low opacity, pale with a strong light leak.

Noir - Includes: Multiply blend, high contrast, very dark with heavy sepia.

Golden - Includes: Soft light blend, warm (with a soft light leak and motion glow).

Cosmic - Includes: Color dodge blend, very low opacity (with a heavy light leak and motion glow).

Dusk - Includes: Difference blend, surreal, moderate amount of grain and moderate amount of light leak.

Selecting one of the above presets will automatically set the corresponding values for all sliders and or toggles; this will also toggle the appropriate blend mode buttons and will trigger a live re-render.

5.2 Frontend Structure (User Interface)

5.2.1 HTML Structure

The application is made using simple and easy to understand HTML5. It is set up in a page style. This page has two areas where you can upload pictures. One area is for Layer I and the other is for Layer II. Layer I is for uploading portrait photographs. Layer II is for textures or other pictures that are used to make the main picture look better. The application also has things like sliders to adjust settings and buttons to choose how to blend pictures together. There are also buttons to save your work and take pictures with your camera. You can compare pictures side by side. There are windows that pop up with more information. There are also some things that you cannot see like areas to upload files and special codes. The application is designed to be easy to use, for people who are not good with computers. It has buttons and clear pictures so it is easy to understand what to do. This makes it a good choice, for places where people can use it.

5.2.2 CSS Styling and Layout

The system is designed like a photographic darkroom with an abundance of color and an overabundance of gold to give it an ‘old time’ feel of a photograph studio. CSS3 has been utilized to ensure that all elements are visually appealing across multiple devices. Many animations have been created, such as the hover state on elements and custom sliders that create ‘motion pictures. The typographic choice is an amalgamation of simple and complex styles to mimic film; the layout has been created to allow for responsive behavior across multiple screen sizes. The designers have created a number of unique effects for the user when the user interacts with the slider to change its color. The combination of a number of small animations, such as countdown timers, has also contributed to the overall enjoyment of using this interactive system. The system also works well on screens, like the ones you find in a kiosk. The look and feel of the system stay the throughout thanks to the custom design. The designers used some tools, like Flexbox and CSS Grid to make the layout responsive. Overall, the system feels like a movie with a lot of visual effects.

5.3 Image Processing Pipeline

5.3.1 Web Worker Architecture

The rendering pipeline does its job inside a Web Worker. This is so it does not block the part of the browser when it is doing complicated things with images. If we did not use a worker to do the rendering the browser would freeze a lot when it is working with images.

The worker gets the images and the render settings through a `postMessage()` call. The images are sent as transferable objects, so the worker takes ownership of them without having to copy them, which saves memory.

Using a worker to do the rendering makes the browser more responsive. This is because the rendering is done in a separate thread, so it does not slow down the main part of the browser that the user interacts with.

5.3.2 OffscreenCanvas Rendering

Inside a Web Worker, `OffscreenCanvas` is extremely helpful for rendering images that don't impact the user's view of the browser. We can perform actions such as changing pixels, blending images together, and adding effects without disrupting the user experience of using the browser.

`OffscreenCanvas` achieves this by processing images through a number of steps. It can turn images to grayscale combine them adjust the contrast add a film grain effect, light leaks sharpen the image and do something called halation rendering.

Using the `OffscreenCanvas` makes rendering images a lot faster than using the standard `Canvas` in the browser. It is an improvement because it does all the hard work in the background, so the user does not have to wait while the image is being processed.

5.3.3 Blend Modes Implementation

In digital compositing, there are 10 commonly used professional blending modes: screen, multiply, overlay, hard light, soft light, lighten, color dodge, color burn, difference and exclusion. Each blending mode is calculated in a floating-point color space $[0,1]$ to produce accurate results. The default mode is 'screen', which can create the bright double exposures seen in analogue cameras

as it adds to the total amount of light in terms of opacity for any overlapping areas of images producing bright results. The multiply blending mode darkens an object, while "overlay" and "hard light" create high contrast combinations with objects in both blending modes being considered on a tonal basis. The amount of blending done depends on the opacity parameter, where the transparency of the new image will replace its original image based on user preset value. When the object has been isolated and a mask been created for blending to take place, the opacity of the mask will also adjust how much blending goes on between the two images as to maintain minimal texture in the background and maximum amount of texture where the subject is extracted.

```
1  const blendFn = {
2    'screen':      (a, b) => 1 - (1 - a) * (1 - b),
3    'multiply':    (a, b) => a * b,
4    'overlay':     (a, b) => a < 0.5 ? 2 * a * b : 1 - 2 * (1 - a) * (1 - b),
5    'hard-light':  (a, b) => b < 0.5 ? 2 * a * b : 1 - 2 * (1 - a) * (1 - b),
6    'soft-light':  (a, b) => {
7      if (b <= 0.5) return a - (1 - 2 * b) * a * (1 - a);
8      const d = a <= 0.25 ? ((16 * a - 12) * a + 4) * a : Math.sqrt(a);
9      return a + (2 * b - 1) * (d - a);
10   },
11   'lighten':     (a, b) => Math.max(a, b),
12   'color-dodge': (a, b) => b >= 1 ? 1 : Math.min(1, a / (1 - b)),
13   'color-burn':  (a, b) => b <= 0 ? 0 : Math.max(0, 1 - (1 - a) / b),
14   'difference':  (a, b) => Math.abs(a - b),
15   'exclusion':    (a, b) => a + b - 2 * a * b,
16   'pin-light':   (a, b) => b < 0.5 ? Math.min(a, 2 * b) : Math.max(a, 2 * b - 1),
17 };
```

5.3.4 Cinematic Effects Pipeline

The rendering engine has special effects that make it look like old film.

These effects include:

1. Sepia toning
2. Film grain that changes with brightness
3. Vignette shading
4. Chromatic aberration

5. Halation glow
6. Sharpening
7. Light leaks
8. Film grain is made using Gaussian noise.
9. This noise is generated using the Box-Muller transform [18], with a luminance-aware amplitude curve $(0.15 + 0.85 \times 4L(1-L))$ so grain peaks at the midtones and is minimal at pure black and white, matching real silver-halide film, and a slight per-channel imbalance (R x 1.00, G x 0.93, B x 0.85) that simulates separate RGB emulsion layers.
10. Halation activates on pixels whose luminance is above roughly 70 percent, extracts a warm highlight channel, blurs it, and screen-blends the warm glow back so bright areas appear to bleed, simulating light bleeding through the film base in real darkroom printing. Sharpening uses a classical unsharp mask [20] (the luminance minus a box-blurred copy of itself, with a radius of about one percent of the image width, added back scaled by the strength setting). Chromatic aberration shifts the red channel a few pixels one way and the blue channel the other while leaving green unshifted, with the shift magnitude scaled by the grain setting to mimic inexpensive lens behavior at high exposures.

This is what happens with camera lenses. All these effects together make the final picture look like it came from a darkroom and was shot on film.

```
1 function addGrain(imageData, intensity) {
2   if (intensity < 0.01) return;
3   const d = imageData.data;
4   const amp = intensity * 54;
5   for (let i = 0; i < d.length; i += 4) {
6     const u = Math.random(), v = Math.random();
7     const gauss = Math.sqrt(-2 * Math.log(Math.max(u, 1e-9))) * Math.cos(2 * Math.PI * v);
8     // Luminance-aware: grain peaks at midtones (4L(1-L)), minimal at pure black/white -
9     // matches real film emulsion where silver halide density is highest at mid-exposure.
10    const luma = (d[i] * 0.2126 + d[i+1] * 0.7152 + d[i+2] * 0.0722) / 255;
11    const curve = 0.15 + 0.85 * (4 * luma * (1 - luma));
12    const n = gauss * amp * curve;
13    // Slight per-channel imbalance simulates separate R/G/B emulsion grain layers
14    d[i] = clamp(d[i] + n * 1.00);
15    d[i + 1] = clamp(d[i + 1] + n * 0.93);
16    d[i + 2] = clamp(d[i + 2] + n * 0.85);
17  }
18 }
```

5.4 AI Segmentation Module

5.4.1 MediaPipe Integration

MediaPipe Selfie Segmentation [10] is utilized by an AI segmentation system, using jsDelivr CDN and run via WebAssembly [14]. It is configured to use `modelSelection: 1` (landscape/general) which should produce higher quality than portrait only. This AI segmentation system separates the subject from the background image. Prior to inference; the source image is downsampled so that the longest dimension is no greater than 640 pixels because there is no added benefit in feeding (to MediaPipe) full resolution images (4K), so incorrect memory usage occurred without improving (in approximately 256 x 256) MediaPipe's internal inference, but would have wasted memory.

```
1  async function getSegmentor() {
2    if (_segmentor) return _segmentor;
3    return new Promise((resolve, reject) => {
4      if (typeof SelfieSegmentation === 'undefined') {
5        reject(new Error('MediaPipe not loaded'));
6        return;
7      }
8      const seg = new SelfieSegmentation({
9        locateFile: (f) => `https://cdn.jsdelivr.net/npm/@mediapipe/selfie_segmentation/${f}`
10      });
11      seg.setOptions({ modelSelection: 1 }); // 1 = landscape/general (better quality)
12      seg.onResults(() => { }); // will be overridden per-call
13      seg.initialize().then(() => { _segmentor = seg; resolve(seg); }).catch(reject);
14    });
15  }
```

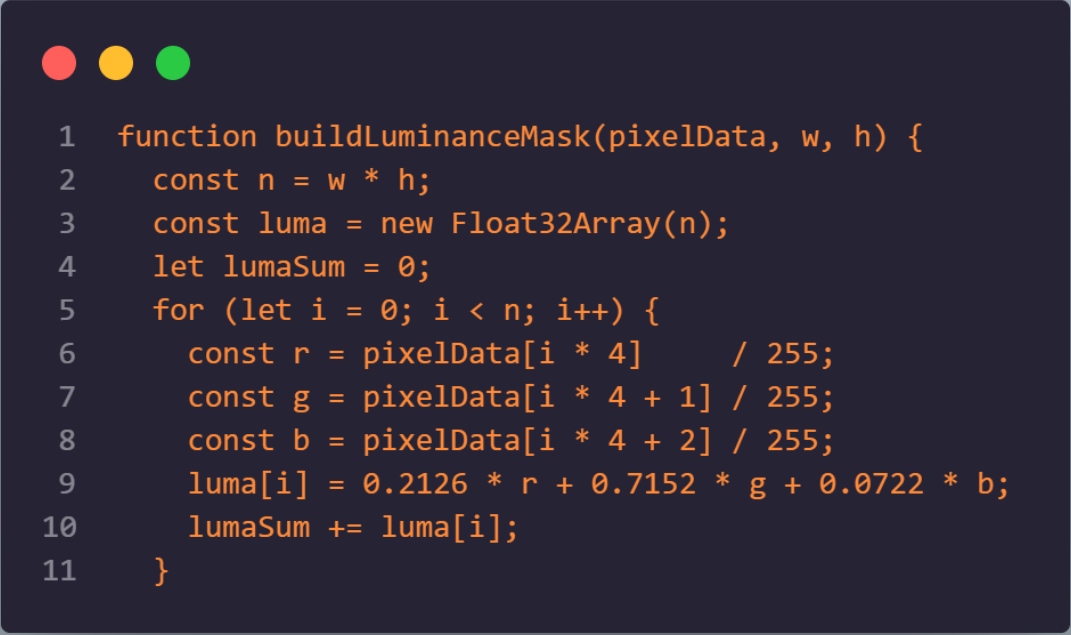
After resolving the segmentation output in this way, the mask created was encoded in the red channel of the canvas produced during the self-segmentation process. When the mask array is accessed from the red channel, it contains pixels with a value between 0 - 255 providing the confidence level for each subject pixel location. Next step is normalizing the mask's pixel data into a float32 array with a range of 0.0 - 1.0 that represents the subject information.

5.4.2 Mask Processing and Smoothing

Segmentation Most segmentation masks seem to have very harsh edges and some minor imperfections around the fine details of items such as hair boundaries. To improve the look of the images the system applies smoothing effects such as Gaussian-based blurring and smoothed-step interpolation. This smooths out the transition zone between the foreground and background areas, thereby enhancing the overall realism of the double exposure blending process.

5.4.3 Luminance Fallback System

The system incorporates a failover option to continue functioning if MediaPipe segmentation is unavailable. More specifically, the fallback approach uses the brightness or darkness of various portions of a captured image to create a mask. A mask will be created by performing a sigmoid threshold calculation based on the brightness value and using a center-weighted prior to assume that the center of the image will be the primary subject. While the algorithm does not generate a mask as accurately as the AI segmentation, the algorithm is capable of separating the primary subject from the background enough that the system will be more reliable in situations where the AI model cannot be loaded.



```
1 function buildLuminanceMask(pixelData, w, h) {
2   const n = w * h;
3   const luma = new Float32Array(n);
4   let lumaSum = 0;
5   for (let i = 0; i < n; i++) {
6     const r = pixelData[i * 4] / 255;
7     const g = pixelData[i * 4 + 1] / 255;
8     const b = pixelData[i * 4 + 2] / 255;
9     luma[i] = 0.2126 * r + 0.7152 * g + 0.0722 * b;
10    lumaSum += luma[i];
11  }
```

5.5 Smartphone Upload System

5.5.1 QR Code Generation

The kiosk makes QR codes on the spot to make it easy for smartphones to connect and upload things from away. The QR code has the website address, a special identifier, for the session and some other important details that are encoded. People use their smartphone cameras to scan the QR code so they can get to the upload screen on their smartphone without having to type in the website address themselves.

5.5.2 Aply Realtime Communication

Real-time communication, between smartphones and the kiosk browser is done using Aply. This uses publish-subscribe channels. Both devices connect to the channels. These channels are identified by kiosk session IDs. The smartphone uploads images in two ways. It can use URLs hosted by ImgBB. It can use compressed Base64 transfer. The method used depends on what's set up and the size limits of the message.

5.5.3 Session Management

The system makes sure that people do not interfere with each other when they use the kiosk at the time. This is done by using session management mechanisms that prevent problems with synchronization and stop someone from using a session that they are not supposed to use. Each time someone uses the kiosk it creates a code that is unique to that kiosk session. If someone leaves the kiosk session it saves the state they were in for a while using something called browser sessionStorage. When people are done with the kiosk session and they disconnect, the way the kiosk and the user talk to each other is closed and any QR code, for the session that was used is no longer good.

5.6 Export and Camera Module

5.6.1 Camera Capture Implementation

The permission dialogue asking users for access to the camera opens the first time the user opens it, creating a more seamless experience for the user. If the user has both their front-facing and rear-facing cameras available, the device will allow them to select which one to use. Once photos have been taken with one of those cameras, they will appear in the user's canvas and then be modified into an image suitable for uploading into the rendering pipeline.

5.6.2 Export Functionality

With the export module, users can use it to export their pictures in several different formats like PNG, JPG, and WEBP. When the user wants to export their picture, the app will create a new version of their picture on a temporary drawing board. The app allows users to add text to their picture, such as a watermark, if desired. The export module is very cool because you can also download your pictures to your mobile phone using a QR code to obtain the pictures from the QR code that the app generates and displays after the export is completed.

5.7 File & Folder Organization

The entire project is organized in file structure that is easy to understand and easy to navigate. This is an example of the file structure:

- `index.html` — Main kiosk interface
- `engine.js` — Core application logic and rendering pipeline
- `style.css` — Styling and responsive layout design
- `remote.html` — Smartphone upload interface
- `run_studio.bat` — Windows deployment and startup script
- `qrcode.min.js` — QR-code generation library

This organization simplifies deployment because the entire project can operate as a standalone browser-based application without external backend dependencies.

DETAILED WORKFLOW EXPLANATION

6.1 Overall User Workflow

The Double Exposure Generator system is designed for easy use and enjoyment inside of a public kiosk environment. Easy as 1-2-3 - a simple enough process for anyone to create a piece of art that they will be proud to display!

Users interface with the Double Exposure Generator system by following a series of steps that begin when the kiosk is powered on, and end when the user retrieves their print. The Double Exposure Generator system utilizes an internet browser to process the images being printed. The Double Exposure Generator system also utilizes artificial intelligence to enhance the prints in a variety of ways. The Double Exposure Generator system creates fun and unique printing experiences, with a variety of cool printing effects similar to those seen on the big screen.

The Double Exposure Generator system is designed and manufactured in such a way that each component and feature is easy to use, ensuring that even non-tech-savvy users have no problem using the system. Thus, the Double Exposure Generator system provides an enjoyable and user-friendly experience for everyone.

6.1.1 Kiosk Session Start

The workflow starts when we launch the application using the deployment batch script. This script starts a local HTTP server and opens the browser interface at localhost:8000. When the kiosk application starts up it finds the local network IP address and sends it to the kiosk application through the website link. When the kiosk application is loaded the user sees the kiosk interface, with areas to upload things, preset cards, rendering controls and export buttons. The kiosk application gets the rendering worker ready loads the things the browser needs and gets the MediaPipe AI segmentation model for when we need it. Now the kiosk application just waits until the user uploads or takes a picture. The kiosk application also starts timers that automatically reset it after a while if nobody is using it.

6.1.2 Image Upload and Capture Flow

People can give us pictures in a few ways. They can upload from their computer drag and drop them send them from their phone or take a picture with their camera. We have two areas where people can upload pictures. One is for Layer I images and the other is for Layer II images.

Typically, Layer I photos depict people; Layer II may depict textures, landscapes, geometric shapes, or auxiliary artwork layered on top of the image. The system collects a photo selected by the user and converts it to an ImageBitmap object so that the system can both display the photo and effectively utilize memory resources during display.

In order for a user to capture a photo using their device's camera, the user must first provide permission for the device to use the camera. The permission process occurs through the use of the MediaDevices API. Once permission has been granted and the photo has been captured, it is displayed on a canvas. Then the captured image is converted to a bitmap object that can be processed by the system for the purpose of displaying the captured image to the user.

After the captured photo has been uploaded, the user is presented with a copy of the captured photo they uploaded. Afterward, the system processes the uploaded photo to identify the different components of the loaded image and combine those components into a new format. The processed uploaded photo can be used by the system for segmentation and compositing operations.

6.1.3 AI Segmentation Workflow

Once a portrait has been uploaded to Layer I of the application, the AI (Artificial Intelligence) component will use AI technology to separate the foreground (the person) from the background of the portrait. This is accomplished with the use of MediaPipe Selfie Segmentation. The portrait will first be resized to a smaller size (to assist in the speed of processing). At this point, MediaPipe performs a segmentation of the image and will generate an output of the location of the person in the image, in the form of a "map". The map is converted to a Float32Array format, allowing it to be efficiently used in the rendering pipeline. In order to improve the overall appearance of the portrait, the map is smoothed and feathered. This process results in softer edges and a more gradual transition between the person and the background. This set of processes is in place to reduce errors and generate an overall more realistic portrait. In the event that the AI segmentation fails or is not

present, an alternative method will be used by the application. This will involve estimating the location of the person through a process of examining the brightness of various regions of the image and using simple techniques to make an educated guess. The completed map is then sent to the rendering worker along with the uploaded images and effect settings, and the image and the map are used together to create the final composited picture.

6.1.4 Rendering and Effects Workflow

The rendering workflow happens inside the Web Worker. This keeps the user interface responsive while the computer does the heavy processing tasks.

When the rendering worker gets image data and rendering settings it starts the compositing pipeline. The rendering process begins by changing the image size to match the chosen output resolution. The system then converts the image to grayscale. Normalizes the tones. After that it applies the chosen blend mode.

The available blend modes are:

- Screen
- Overlay
- Multiply
- Soft Light
- Hard Light
- Difference

The blend mode helps combine the images in ways. The system uses the chosen blend mode to create an image. Next special effects are added to the image one after another.

These effects include:

- Making the image contrast stronger
- Adding a sepia tone to the image
- Simulating film grain
- Shading the image with a vignette
- Creating chromatic aberration

- Adding a glow around areas
- Sharpening the image
- Rendering light leaks

The film grain effect is created randomly using a noise pattern. This makes the image look like its made from film. The halation effect makes it look like light is bleeding around areas in the image. This is like what happens in film when its overexposed. The chromatic aberration effect shifts the color channels. This makes the image look like it was taken with a lens that has imperfections. This is a look in cinematic photography. When the rendering is complete the final image is sent back, to the browsers thread. It is then displayed inside the preview canvas. The final image is a combination of the image and the effects that were added. The user can see the image in the preview canvas.

6.1.5 Export and Download Flow

When the rendering is finished users can save the picture they made in formats, like PNG, JPG and WEBP. The system makes the picture again at the size the user chose. Then it turns the picture into a file that the user can download. Users can also add a watermark to the picture if they want to. The system puts some text on the picture before making the file. Users can save the picture to the kiosk or they can send it to their smartphone by using a special code. If they choose to send it to their phone the system puts the picture online. Makes a special code that has the link to download the picture. This is the step that users do when they use the kiosk system.

6.2 Smartphone Upload Workflow

6.2.1 QR Code Pairing Process

The smartphone upload process starts when the kiosk makes a special code called a QR code. This QR code has the website address for uploading files and a unique identifier. The kiosk shows this QR code on its screen so people can scan it. People use their smartphone cameras to scan the QR code. When they do this, it automatically opens the upload page, on their phones internet browser. Then the smartphone connects to the communication channel that the kiosk uses, which is called Aply Realtime. Using a QR code to connect the smartphone to the kiosk is a way to do it. The

smartphone upload process does not need people to set up the network or type in any addresses. The QR code makes it easy for the smartphone and kiosk to talk to each other.

6.2.2 Image Transfer Process

Once paired and ready for use, users have the ability to select images stored on their mobile device. The mobile interface compresses these images prior to being sent (transmitted) via Ably Realtime channels. Users can transmit either Base64 encoded or ImgBB URLs based on settings and file size. Once the image is received by the kiosk browser, it will be reconstructed and displayed immediately allowing users to send images from their mobile device without touching the kiosk.

6.2.3 Session End and Reset

The completion of an upload, or a state of inactivity created by a user, may cause the kiosk session to close manually or as a result of a timer. Upon closure of the session, the kiosk will subsequently disconnect from the Ably channel and invalidate its QR session identifier. The device will then display a message on its smartphone interface informing the user that their session has ended, as well as disabling uploads, and clearing the sessionStorage of the device. At this point, the kiosk will return to an idle state, which allows it to accept new users.

6.3 Rendering Pipeline Workflow

6.3.1 Web Worker Communication Flow

The rendering process begins when the main browser thread communicates with a Web Worker to send ImageBitmap objects along with parameters using the postMessage() method. Transferable objects enable an application to transfer ownership of images directly, preventing duplicates in memory. The Web Worker will create an OffscreenCanvas context and perform the image processing sequentially. When image processing has been completed, the final bitmap is sent to the main thread for display/export with improved performance through separation of rendering and interface interaction.

6.3.2 Blend Mode Processing Steps

After the processes of normalization and alignment have been completed, the blending of images begins. The system then transforms each of the image channels into normalized floating-point representations for use in blend mode calculations. Each of the different blend modes has its own unique set of mathematical operations, such as: "Screen" produces a brighter image, "Multiply" produces a darker image, while "Overlay" produces an enhanced contrast image whilst maintaining the same brightness rating. The final image produced will then go through an opacity control process before proceeding to enter into the cinematic effects phase.

6.3.3 Cinematic Effects Processing Steps

The "Cinematic Effects" stage uses a series of sequentially applied filters for post-processing to emulate the look of analog photography. Initially, the tonal adjustments occur as enhancements to overall image contrast and sepia toning. The next step in the process involves creating the appearance of film grain using Gaussian noise based on luminance. After film grain is created, additional post-processing occurs to simulate the appearance of vignetting, which darkens the image corners and draws the viewer's eye to the center of the image; halation, which creates a 'soft' look around the highlight areas; and chromatic aberration, which creates channel offsets between RGB channels to simulate lens imperfections. Finally, the image is sharpened with edge contrast enhancement filters before being displayed as an image.

Conclusion

The Double Exposure Generator has shown that it is feasible to use intelligence in creating artistic photographically double-exposed images through a browser-based kiosk. The system does not require any server support; all of the image processing occurs on the client-side by using the browser as a conduit between images and intelligence sources to separate image contents, communicate via real-time to smartphones, and include applied special effects to the combined resultant images. This all has been accomplished by a single application that allows individuals to use this creative experience at various locations.

The Double Exposure Generator project has achieved its goal of successfully implementing rendering pipelines in its web technologies by utilizing a number of advanced technical techniques (HTML5, CSS3, and JavaScript). Also included were use of specific technologies such as Web Workers, OffscreenCanvas, and ImageBitmap to ensure fast image processing and smoother interface interaction for users within all environments. In addition, MediaPipe Selfie Segmentation has been employed for the identification and separation of the subject from the background in an image, which fosters the future use of this technology by those who are not very computer/technology advanced users. Furthermore, methods for the combination of images and the inclusion of applied special effects, including sepia tone, film grain, and vignette shading effects, provided creative options within each of the combined images.

People created a quick and simple way to upload photographs from their smartphones by using QR codes, using a kiosk design so people could upload, process & download their photographs quickly and easily through the Internet.

The Double Exposure Generator Project has a few constraints; one of those constraints is that you need a live Internet connection. Another constraint is that the kiosk has to be run on a modern web browser. Lastly, the Double Exposure Generator Project demonstrates that the advanced capabilities of modern web browsers can be utilized for many advanced applications without the aid of a specific server.

The Double Exposure Generator Project is a creative use of technology to assist in the creative process. Additionally, the Double Exposure Generator Project clearly demonstrates the capabilities of web technologies (e.g., photo sharing applications, photo editing applications) to create multimedia applications.

REFERENCES

- [1] Adobe Inc., "Photoshop User Guide," 2024. [Online]. Available: <https://helpx.adobe.com/photoshop/user-guide.html>
- [2] Mozilla Developer Network, "MediaDevices.getUserMedia()," MDN Web Docs, 2024. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia>
- [3] A. Adams, *The Negative*. Boston, MA, USA: Little, Brown and Company, 1981.
- [4] Google, "Snapseed Help Center," 2024. [Online]. Available: <https://support.google.com/snapseed>
- [5] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Upper Saddle River, NJ, USA: Pearson, 2018.
- [6] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *Proc. Med. Image Comput. Comput. Assist. Interv. (MICCAI)*, 2015, pp. 234-241.
- [7] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2015, pp. 3431-3440.
- [8] L.-C. Chen, G. Papandreou, I. Kokkinos, K. Murphy, and A. L. Yuille, "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 4, pp. 834-848, 2018.
- [9] A. Howard et al., "Searching for MobileNetV3," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2019, pp. 1314-1324.
- [10] C. Lugaresi et al., "MediaPipe: A Framework for Building Perception Pipelines," *arXiv preprint arXiv:1906.08172*, 2019.
- [11] Mozilla Developer Network, "Canvas API," MDN Web Docs, 2024. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API
- [12] Mozilla Developer Network, "Using Web Workers," MDN Web Docs, 2024. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API/Using_web_workers
- [13] Mozilla Developer Network, "OffscreenCanvas," MDN Web Docs, 2024. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/API/OffscreenCanvas>
- [14] A. Haas et al., "Bringing the Web Up to Speed with WebAssembly," in *Proc. 38th ACM SIGPLAN Conf. Program. Lang. Design Implement. (PLDI)*, 2017, pp. 185-200.
- [15] I. Fette and A. Melnikov, "The WebSocket Protocol," IETF, RFC 6455, Dec. 2011. [Online]. Available: <https://tools.ietf.org/html/rfc6455>
- [16] Ably Realtime, "Ably Realtime Platform Documentation," 2024. [Online]. Available: <https://ably.com/docs>
- [17] ISO/IEC 18004:2015, "Information technology - Automatic identification and data capture techniques - QR Code bar code symbology specification," International Organization for Standardization, Geneva, Switzerland, 2015.

- [18] G. E. P. Box and M. E. Muller, "A Note on the Generation of Random Normal Deviates," *Ann. Math. Statist.*, vol. 29, no. 2, pp. 610-611, 1958.
- [19] ITU-R Recommendation BT.709-6, "Parameter values for the HDTV standards for production and international programme exchange," International Telecommunication Union, Geneva, Switzerland, 2015.
- [20] R. C. Gonzalez and R. E. Woods, "Spatial Filtering," in *Digital Image Processing*, 4th ed., ch. 3, Upper Saddle River, NJ, USA: Pearson, 2018.